Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M1 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

Abstract*—Modern data pipelines increasingly depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-crafted declarative rules. Recent work has started to introduce machine learning (ML) anomaly detectors inside ETL workflows, but systematic, apples-to-apples comparisons between rule-based, classical ML, and deep learning approaches remain scarce—especially when evaluated within a realistic pipeline and broken down by the type of anomaly being injected. This paper presents an open-source ETL pipeline in which anomaly detectors from different families share one `fit/predict/score` interface and are scored on the same pipeline path: statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM), with deep learning detectors (Autoencoder, LSTM-Autoencoder) planned behind the same interface. We evaluate the detectors on the Kaggle Credit Card Fraud dataset and on a controlled synthetic dataset generated with Faker in which six distinct anomaly types are injected at controlled rates. Beyond aggregate precision, recall, F1, ROC-AUC and PR-AUC, we report a per-anomaly-type detection rate that maps each method to the anomaly categories it actually catches. The aggregate metrics make the families look interchangeable; the per-type view shows each has a narrow, largely non-overlapping competence, so broad coverage needs several families rather than the single best one. Our goal is a reproducible, pipeline-integrated benchmark that answers the practical question: *which family of detectors should I use, for which kind of data quality problem, at what operational cost?

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications [1]. The value delivered downstream is bounded by the quality of the data that reaches the destination, which is why data quality has long been a first-class concern for data engineers. Poor data quality has been estimated to cost organizations an average of $12.9M per year [2], making automated, continuous quality control a critical capability rather than an optional one.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [3]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [2], [4], [5]. The existing literature, however, suffers from three gaps.

First, most published pipelines are conceptual architectures [2], [4] that do not ship with reproducible code or quantitative comparisons. When experiments are reported, they typically exercise a single algorithm on a single dataset—for instance, Isolation Forest on Credit Card Fraud [5], which achieved a near-random ROC-AUC of 0.51 and thus offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of dozens of detectors—but these benchmarks evaluate algorithms *in vacuum*, outside of an ETL context. They do not consider the declarative rule-based baselines that dominate production systems, and they do not measure end-to-end pipeline integration costs (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors excel at different kinds of anomalies. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers [8]; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Contributions.: This paper addresses these three gaps. We present:

1) **A reproducible ETL pipeline** in which rule-based and classical ML anomaly detectors share one interface and are scored on the same extract–transform–detect path, released as open-source code, with deep-learning detectors planned behind the same interface.
2) **A head-to-head benchmark** of statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM) on Credit Card Fraud and on a controlled synthetic dataset, reporting precision, recall, F1, ROC-AUC, PR-AUC, and in-pipeline latency.
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at controlled rates, we produce a method-by-type detection-rate matrix. The families turn out to have narrow, largely non-overlapping competence that the aggregate metrics hide.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes our pipeline architecture, the detectors we compare, and the datasets and metrics we use. Section IV reports experimental results. Section V discusses when rules, classical ML, and deep learning are each the right tool, and highlights practical trade-offs in latency and operational cost. Section VI concludes and outlines future work.

II. Related Work

To be written. Planned subsections:

- Data quality in ETL pipelines (Khatri & Brown [1]; Data Quality Survey [9])
- Rule-based and declarative approaches (Great Expectations, Deequ, Soda [3])
- Classical ML for anomaly detection (Isolation Forest, LOF, One-Class SVM; Goldstein & Uchida [8]; PyOD [7])
- Deep learning (Autoencoder, LSTM-Autoencoder)
- Existing benchmarks: ADBench [6] and why it does not cover rule-based baselines, ETL integration, or per-anomaly-type analysis
- ML in ETL pipelines: Gopa [2], Marupaka & Rangineni [4], Sokkula & Vuppala [5] as illustrative limitations

III. Methodology

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and deep models can later be dropped in without changing the harness.

B. Rule-based detectors

This stage compares three rule-based baselines. All three define a band of "normal" values per column and flag a row when it leaves the band on at least one column; they differ in how the band is estimated and in how robust that estimate is to the very outliers being sought.

Z-score standardizes each numeric column on the fitted data and flags a row when any column exceeds $|z| > 3$. The threshold follows the empirical rule: under an approximately normal distribution about 99.7% of values lie within 3σ, so beyond it is rare by construction. Its weakness is that the mean and standard deviation are themselves inflated by extreme values, so a large outlier can mask others.

IQR learns Tukey's fences $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ per column, where the interquartile range spans the central 50% of the data, and flags a row falling outside any fence. Quartiles are robust: unlike the mean and standard deviation they barely move when extreme values are added, so the fences are not distorted by the outliers they are meant to catch. The 1.5 multiplier corresponds to about 2.7σ under normality, isolating a comparable tail.

Great Expectations encodes a declarative expectation suite [3] and makes no distributional assumption: a not-null expectation on every column and a range expectation per numeric column, with bounds set to the 0.1th and 99.9th percentiles of the fitted data. Using inner percentiles rather than the observed min/max is what turns the range check into a usable rule instead of a no-op on the fitting data. A row is flagged when it violates at least one expectation; its score is the fraction of the suite it violates.

The continuous scores of the statistical detectors are the worst standardized deviation (Z-score) and the largest fence exceedance in IQR units (IQR). The axis of difference between the three is thus robustness and the assumption made about the shape of the data, which is precisely what the learned models of the next stage are meant to handle better.

C. Classical machine-learning detectors

The second family replaces the per-column band with a learned model of the joint feature space, so that a row can be flagged for an implausible *combination* of otherwise-normal values. Three unsupervised detectors are taken from PyOD [7] rather than reimplemented: *Isolation Forest*, which scores a row by how few random axis-aligned splits are needed to isolate it; *Local Outlier Factor* (LOF), which compares a point's local density to that of its neighbours; and a *One-Class SVM*, which fits a boundary enclosing the bulk of the training data. All three receive the same preparation: non-numeric columns are dropped, missing cells are median-imputed, and every feature is standardized, since LOF and the One-Class SVM are distance-based. The PyOD `decision_function` provides the continuous score; the boolean flag uses a contamination of 0.10, so each model marks its top 10% most anomalous

rows. Contamination sets only the flag threshold and leaves ROC-AUC and PR-AUC unaffected.

D. Synthetic per-anomaly-type dataset

The real datasets carry a single binary label that mixes every kind of defect together, so they cannot say *which* problem a detector is good at. We therefore generate a controlled dataset of 12,360 customer records with Faker and NumPy, in which income is correlated with balance and transaction size and account age is spread over three years. Into this clean population we inject six anomaly types, each at a rate of 3% and each row tagged with its category: *missing values* (one or two cells nulled), *exact duplicates* (a clean row copied), *univariate outliers* (one numeric field pushed far out of range), *multivariate outliers* (every field individually in range but the combination implausible, e.g. an 18-year-old with a top-percentile balance and income), *temporal drift* (recent-signup rows whose income and balance distribution has shifted upward by half), and *invalid formats* (malformed e-mail, out-of-domain country code, impossible age). Because the category of every injected row is known, we report a detection rate per method and per type.

E. Datasets and metrics

The real-data results below use the Credit Card Fraud dataset (284,807 transactions, 30 numeric features, 0.17% labelled fraud), a standard and strongly imbalanced benchmark. Each detector is fitted and evaluated on the feature matrix, and its output is scored against the ground-truth label. Because the detectors emit both a threshold decision and a graded score, we report threshold metrics (precision, recall, F_1) alongside two threshold-independent ranking metrics, ROC-AUC and PR-AUC. On a dataset as imbalanced as this one PR-AUC is the more informative: the false-positive rate underlying ROC-AUC is diluted by the large pool of true negatives and stays optimistic, whereas precision reacts directly to false alarms. We report both, however, because the appropriate metric depends on each dataset's base rate — PR-AUC's random baseline equals the prevalence and is therefore not comparable across datasets, while ROC-AUC's is always 0.5 and is. As further datasets are added we keep both and read the one suited to each dataset's balance. We also report the fraction of rows flagged and wall-clock time. On the synthetic dataset the same aggregate metrics are reported, complemented by the per-anomaly-type detection rate of Section IV-C. The deep-learning detectors (Autoencoder, LSTM-Autoencoder) are the object of the next stage.

IV. Experiments and Results

A. Real fraud data: rules and Isolation Forest

Table I reports the detectors on Credit Card Fraud. The picture is consistent across metrics: every method recovers most frauds (recall 0.71–0.97) but at unusable precision (≤ 0.04), because neither a fixed statistical rule nor an unsupervised model tuned to flag 10% of rows can separate a 0.17% positive class from the long tails of legitimate traffic. IQR is the most aggressive, flagging almost half of all rows, which collapses its PR-AUC. The declarative Great Expectations suite is the most selective (3% of rows) and obtains the best PR-AUC; Isolation Forest obtains the best ROC-AUC and improves PR-AUC over both statistical rules, at 0.157, without reaching the declarative suite. Measuring detection inside the pipeline also exposes a cost gap: the Great Expectations suite is about 60× slower than the statistical rules (13.2 s vs 0.2–0.6 s on the full dataset). Local Outlier Factor and the One-Class SVM are omitted from this table: LOF scoring is superlinear in the number of rows scored and the One-Class SVM has quadratic training, so neither runs at this dataset's scale inside the pipeline. They are compared with the other methods on the smaller synthetic dataset below, which is itself a pipeline-integration finding.

TABLE I
DETECTORS ON CREDIT CARD FRAUD (FULL DATASET). HIGHER IS BETTER EXCEPT FOR FLAGGED ROWS AND TIME. TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY. ISOLATION FOREST IS FITTED ON A 20,000-ROW SAMPLE AND SCORED ON ALL 283,726 ROWS.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.899	0.022	0.944	0.090	13.4	**0.2**
IQR	0.003	**0.968**	0.007	0.922	0.020	48.6	0.6
Great Expectations	**0.039**	0.710	**0.075**	0.848	**0.206**	**3.0**	13.2
Isolation Forest	0.014	0.882	0.028	**0.949**	0.157	10.4	3.2

TABLE II
ALL DETECTORS ON THE SYNTHETIC DATASET (12,360 ROWS, 17.5% ANOMALOUS). TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY.

Detector	Prec.	Rec.	F_1	ROC	PR	Time (s)
Z-score	**1.000**	0.188	0.316	0.733	0.540	**0.003**
IQR	0.581	**0.402**	**0.475**	0.677	0.451	0.005
Great Expectations	0.952	0.238	0.381	0.618	0.361	0.893
Isolation Forest	0.599	0.343	0.436	**0.738**	0.505	0.528
LOF	0.529	0.269	0.356	0.619	0.394	0.412
One-Class SVM	0.583	0.334	0.425	0.712	**0.512**	9.826

B. Synthetic dataset: aggregate metrics

Table II reports all six detectors on the synthetic dataset (12,360 rows, 17.5% anomalous). On aggregate metrics the three families look interchangeable: F1 ranges only from 0.32 to 0.48 and PR-AUC from 0.36 to 0.54, with no family strictly ahead. Z-score reaches perfect precision by firing only on the most extreme rows; Isolation Forest and the One-Class SVM lead PR-AUC among the learned models. The One-Class SVM again pays for it in latency (9.8 s, about 20× Isolation Forest). Read on their own, these numbers would suggest the choice of detector barely matters. The per-type breakdown shows the opposite.

C. Synthetic dataset: detection rate by anomaly type

Fig. 1 gives the recall of each method on each injected anomaly type. The structure that the aggregate table hid is now visible: every method has a narrow competence.

- *Univariate outliers* are caught by every distributional method (recall 0.91–0.99), and only by those.

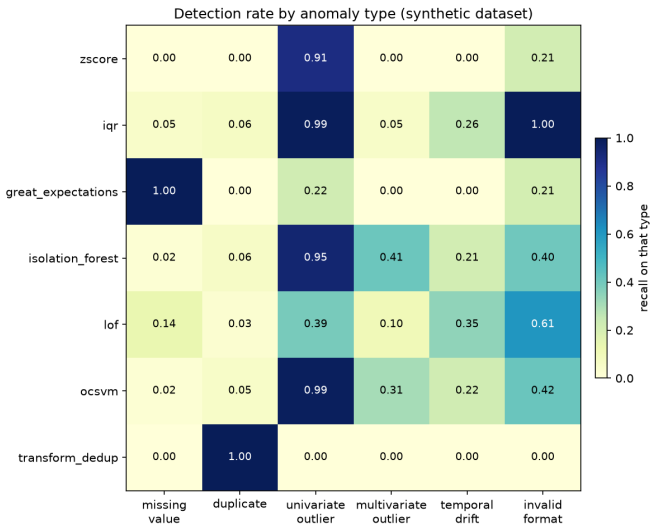


Fig. 1. Detection rate (recall) of each method on each injected anomaly type, synthetic dataset. Darker is higher. `transform_dedup` is the deduplication rule of the transform stage, shown for reference.

- *Missing values* are caught only by the Great Expectations not-null expectations (1.00 against ≤ 0.14 for every other method), because the statistical rules propagate `NaN` to a non-flag and the learned models see imputed values.
- *Exact duplicates* are invisible to every detector: a copy of a normal row looks normal. The deduplication rule in the transform stage is the only thing that catches them, which is the argument for keeping that stage.
- *Multivariate outliers* separate the families: Isolation Forest (0.41) and the One-Class SVM (0.31) are the only methods with non-trivial recall, since they are the only ones that model the joint distribution.
- *Invalid formats* are caught mainly through the impossible numeric values they carry (IQR 1.00 via the out-of-range age); the malformed e-mail and country code are missed by all current detectors.
- *Temporal drift* is the weakest column for every method (best recall 0.35, LOF), because the shifted values still fall inside the global range and no detector uses the timestamp.

The practical reading is a short decision list: null-rate problems go to declarative rules, single-column outliers to any statistical rule, implausible combinations to a tree- or boundary-based model, and duplicates and drift to dedicated stages that none of these detectors replace.

V. Discussion

a) Rules and learned models are complementary, not competing.: The per-type heatmap shows almost no overlap between what the declarative rules catch (missing values), what the statistical rules catch (single-column outliers), and what the learned models add (implausible combinations). A pipeline that wants broad coverage needs more than one family, not the single best one. This is a different conclusion from aggregate benchmarks, where a single ranking invites picking one winner.

b) Aggregate metrics on a mixed label are misleading.: On both datasets the aggregate table compresses very different behaviour into similar F1 and PR-AUC values. The Credit Card Fraud label mixes every defect type together, so a method that is excellent on one kind of anomaly and blind to the rest scores the same as a mediocre all-rounder. The controlled dataset is what makes the difference measurable.

c) Pipeline integration changes the shortlist.: Latency and scaling are not secondary. Great Expectations is accurate but $60\times$ slower than the statistical rules; the One-Class SVM has quadratic training and Local Outlier Factor has superlinear scoring, so neither survives contact with a full-size dataset in the pipeline. A benchmark that scores algorithms in isolation would not surface this.

d) Threats to validity.: The synthetic dataset encodes our own assumptions about what each anomaly type looks like, and the injection rates and magnitudes are choices. The ML detectors use a fixed contamination of 0.10 rather than a tuned operating point, which depresses their precision on the highly imbalanced fraud data. Results on a second real dataset are needed before the per-type ranking can be presented as general.

VI. Conclusion

We built an ETL pipeline in which rule-based and classical machine-learning anomaly detectors share one interface and are scored on the same path, and a controlled dataset that labels six anomaly types separately so detection can be measured per type. On aggregate metrics the detector families are close on both Credit Card Fraud and the synthetic dataset. The per-type view is where they diverge: declarative rules own missing values, statistical rules own single-column outliers, tree- and boundary-based models are the only ones that catch implausible combinations, and duplicates and temporal drift are caught by neither and need dedicated handling. Latency and scaling further constrain which detector can sit in a real pipeline.

The immediate next steps are the deep-learning detectors (Autoencoder, then LSTM-Autoencoder) behind the same interface, a second real dataset (UCI Water Quality) to test whether the per-type ranking generalises, a contamination set from each dataset's prevalence, and a detector that uses the timestamp so temporal drift stops being an empty column.

References

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

[2] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, 2025.

[3] Various Authors, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[4] P. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, 2024.

[5] S. R. Sokkula and S. R. Vuppala, “Implementing machine learning for ETL data transformation and anomaly detection,” *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, “ADBench: Anomaly detection benchmark,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, “PyOD: A python toolbox for scalable outlier detection,” *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019.

[8] M. Goldstein and S. Uchida, “A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data,” *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] Various Authors, “A survey on data quality dimensions and tools for machine learning,” *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614